IMPROVED SYMBOLIC DRUM STYLE CLASSIFICATION WITH GRAMMAR-BASED HIERARCHICAL REPRESENTATIONS

Léo Géré[1] **Nicolas Audebert**[1,2] **Philippe Rigaux**[1]
[1] Conservatoire national des arts et métiers, CEDRIC, F-75141 Paris, France
[2] Univ. Gustave Eiffel, ENSG, IGN, LASTIG, F-94160 Saint-Mandé, France
leo.gere@lecnam.net, nicolas.audebert@ign.fr, philippe.rigaux@cnam.fr

ABSTRACT

Deep learning models have become a critical tool for analysis and classification of musical data. These models operate either on the audio signal, *e.g.* waveform or spectrogram, or on a symbolic representation, such as MIDI. In the latter, musical information is often reduced to basic features, *i.e.* durations, pitches and velocities. Most existing works then rely on generic tokenization strategies from classical natural language processing, or matrix representations, *e.g.* piano roll. In this work, we evaluate how enriched representations of symbolic data can impact deep models, *i.e.* Transformers and RNN, for music style classification. In particular, we examine representations that explicitly incorporate musical information *implicitly* present in MIDI-like encodings, such as rhythmic organization, and show that they outperform generic tokenization strategies. We introduce a new tree-based representation of MIDI data built upon a context-free musical grammar. We show that this grammar representation accurately encodes high-level rhythmic information and outperforms existing encodings on the GrooveMIDI Dataset for drumming style classification, while being more compact and parameter-efficient.

1. INTRODUCTION

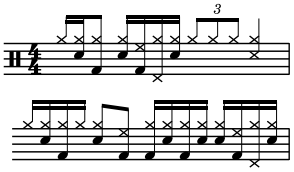
In the last few years, machine learning (ML) has significantly changed how the Music Information Retrieval (MIR) community deals with tasks such as style and composer classification, music generation, pitch and rhythm detection, etc. Yet, training deep learning models on music raises the question of the representation of this data. Depending on the input format (audio, MIDI, musical score…), different representations, *i.e.* different *encodings*, are possible. Each encoding has advantages and drawbacks: some representations, *e.g.* waveforms, focus on raw low-level acoustic features, while others, *e.g.* sheet music, encode high-level abstract semantics of the musical language.

While deep neural networks had great success on audio signal, *i.e.* waveforms and spectrograms, machine learning for symbolic MIDI remains understudied. In this work, we seek to build effective representations of symbolic music, with a focus on recorded MIDI performances. Multiple possible representations of MIDI music coexist in the literature. Most of them contains only low-level information, such as the timing of the onset and the offset of each note and their velocity. This is due to practical constraints: typical MIDI recordings usually do not contain any information about tonality, tempo, time-signature or rhythm. Hence, a model trained on such MIDI samples typically needs to allocate a part of its weights to extract these relevant high-level features from the data. Building better representations of MIDI data to encode semantic musical information could therefore be beneficial to the training of deep models and their efficiency, as they could directly focus on using these features rather than extracting them from the data first.

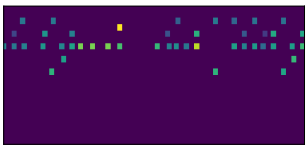
Music classification has been a task of choice for MIDI performances. Preliminary works from [1] in 2007 encoded MIDI as strings and used Kolmogorov complexity to compare music pieces. [2] introduced jSymbolic, a library to extract high level features from MIDI files, such as pitch histograms, a line of work extended by `music21` [3] and `musif` [4]. As new MIDI datasets have been introduced for composer [5] and style classification [6], efforts have been made to evaluate how MIDI representations affect deep models. [7] introduced MidiTok, a tokenization framework to encode MIDI files as a sequence of tokens, suitable for Transformers and Recurrent Neural Networks (RNN). More recently, [8] compared different neural architectures for various MIDI encodings: Convolutional Neural Networks (CNN) trained on Piano rolls, Transformers trained on sequences of tokens, and Graph Neural Networks (GNN) trained on graphs extracted from MIDI files.

In this line of work, we aim to design a representation of MIDI files that is both efficient and discriminative for classification tasks, by incorporating high level musical information directly in the preprocessing. To do so, we explore a new representation based on the *rhythmic tree* structure, built from a context-free grammar tailored to symbolic music. We show that this representation outperforms existing encodings, such as tokenizations or piano rolls, on a drumming style classification built upon the GrooveMIDI Dataset [6]. In addition, our rhythmic tree-based encoding results in smaller deep models, with less parameters, able to be trained on less data compared to existing representations.

© L. Géré, N. Audebert, and P. Rigaux. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** L. Géré, N. Audebert, and P. Rigaux, "Improved symbolic drum style classification with grammar-based hierarchical representations", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.



(a) Base score



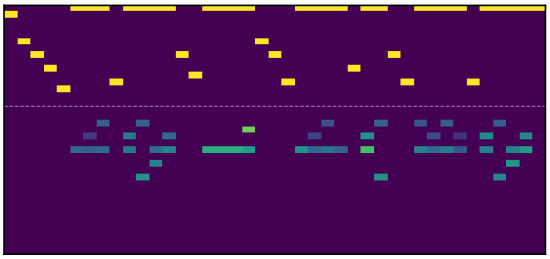
(b) Piano roll

```
DrumOn_42, Velocity_43,
TimeShift_0.1.8,
DrumOff_42,
TimeShift_0.1.8,
DrumOn_42, Velocity_39,
DrumOn_38, Velocity_23,
TimeShift_0.1.8,
DrumOff_42, DrumOff_38...
```

(c) Tokenization (MIDI-Like)

```
(p=42,v=0.35,d=0.1,t=0.000),
(p=42,v=0.32,d=0.1,t=0.175),
(p=38,v=0.17,d=0.1,t=0.006),
(p=36,v=0.32,d=0.1,t=0.181),
(p=42,v=0.35,d=0.1,t=0.017),
(p=42,v=0.48,d=0.1,t=0.358),
(p=38,v=0.41,d=0.1,t=0.034),
(p=46,v=0.51,d=0.1,t=0.158)...
```

(d) Note Tuples



(e) Linearized Rhythmic Tree

Figure 1: Different representations of the same two bars of drums. Score (a) is present for reference only.

2. BACKGROUND

2.1 MIDI Representations

MIDI is a lightweight musical information exchange format. It does not carry audio data, but only timestamped events, *e.g.* a note being played, featuring its pitch and velocity, a note being released, a pedal change, etc. It is suitable for recording as it captures the performer's expressiveness, but does not require metadata that are found in a score, such as tempo, time-signature [1], tonality and voices [9]. We discuss below the most common MIDI representations for ML.

2.1.1 Piano Roll

The piano roll is a visual representation of MIDI files inspired by the analog rolls for piano players. It consists in a 2D matrix with one dimension for pitches, and one for time. A note at pitch p with a `NOTE_ON` event at x_{on} and `NOTE_OFF` event at x_{off} is given a positive value at positions $(x, p)_{x \in [x_{\text{on}}, x_{\text{off}}]}$, as shown in Figure 1b. Often, the value in a matrix cell is one of the properties of the MIDI event, *e.g.* the velocity. This representation is popular, as its 2D structure allows to easily adapt deep models inspired by image processing (*e.g.* CNN) to music tasks [10–12]. However, it can result in large sparse matrices with many zeros, since the time dimension must be discretized with a time step smaller than the shortest MIDI event. In addition, piano rolls tend to be very long and redundant, since many successive vectors will be identical.

[1] MIDI recordings can contain tempo and time-signature, but only through manual addition *a posteriori*.

2.1.2 Sequence of Tokens or Notes

Similar to Natural Language Processing (NLP) techniques, recent works have adopted sequence-like representations, especially suitable for RNN and Transformers architectures. They encode MIDI files as sequences of events. These events are in turn transformed into *tokens*, *i.e.* discrete values from a vocabulary V. Many tokenizations exist, some consisting in a simple token/event mapping with MIDI files (MIDI-Like [13, 14], see Figure 1c), while others include note durations (Structured [15], TSD [16]). More sophisticated tokenizers include higher level information about bar and position in the bar, such as REMI [17].

Finally, MIDI files can be represented as "note tuples", *i.e.* sequences of notes with attributes. For example, [18] represents each note by a set of four values: pitch, velocity, duration and time-shift compared to the previous note (cf. Figure 1d). This representation is much more compact than piano rolls or sequence of tokens.

2.2 Formal Grammar

This work designs a symbolic music representation for deep networks based on a grammar-based rhythmic tree. As a starting point, a formal grammar defines the syntax of a language L. It consists in a set of symbols, associated with production rules used to rewrite non-terminal symbols into other (non-)terminal symbols. Applied successively, those rules can produce every possible sentence of L.

2.2.1 Context-Free Grammar

Succinctly, a context-free grammar [19] is a type of formal grammar for which the production rules do not depend on other context than the left-hand-side symbol. It is defined as a 4-tuple $G = (V, \Sigma, R, S)$. V is a finite set of non-terminal symbols, including the special *start symbol* S. Σ is a finite set of terminal symbols, called the alphabet. Finally, R is a finite set of production rules of the form $a \to b$, where $(a, b) \in V \times (V \cup \Sigma)^*$ in which * denotes the Kleene star operator, *i.e.* a pattern repeated of 0, 1 or more times.

The application of a sequence of rules can be represented as a tree, in which the parent node is represented by the left-hand-side of each rule, and the child nodes are the symbols on the right-hand-side. Once every non-terminal symbol has been resolved into a terminal symbol, we obtain a *parse tree* representing the structure of a sentence of L according to G, with elements of Σ as leaves, and S as root.

2.2.2 Musical Grammar

In a homophonic musical score (monophonic voice that can include chords [20]), rhythm can be represented as a tree [21, 22]. For example, in a 4/4 music piece, a measure could be split into two half notes. Then, each half note can be further divided into two quarter notes, or into a triplet of quarter notes, etc. The `qparse` library [23] is a MIDI-to-score transcription framework that produces a sheet music by parsing a MIDI file with a weighted context-free grammar and dynamic programming, with applications *e.g.* to automatic drum transcription [24]. While designed for a handcrafted music transcription algorithm, the intermediate

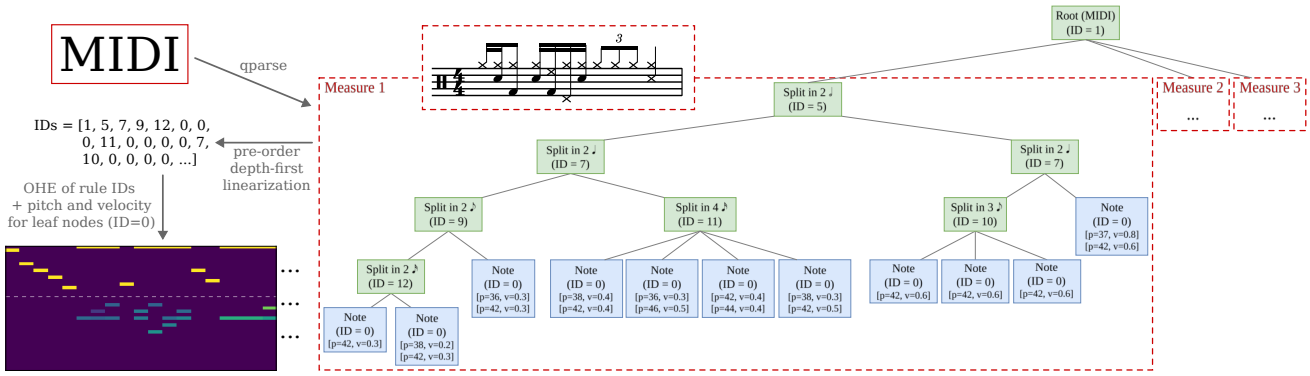


Figure 2: Example of tree built by `qparse` after rules simplification and re-rooting of measures (right), with its associated linearization and vector representation (left). In the matrix, the part above the dashed line contains the one-hot encoded rules (blue/yellow for 0/1), and the one below contains the playing instruments for terminal nodes (color representing velocity).

parsing tree computed by `qparse` contains rich rhythmic information that is also valuable as an input to deep models. Note that, while we our work uses `qparse` to obtain rhythmic trees, our contribution lies in evaluating this tree representation of music, regardless of its construction. We expect our representation to generalize to other parsers.

3. METHODOLOGY

3.1 Linearized Rhythmic Tree

To build our high-level MIDI representation, we linearize a rhythmic tree obtained using a context-free grammar, enriched by information about pitch and velocity in leaves. We call this representation Linearized Rhythmic Tree (LRT). To achieve this goal, we leverage the transcription framework `qparse` [23] to extract its internal intermediate rhythmic tree representation. Note that we only consider homophonic inputs since this is what `qparse` MIDI grammar supports. `qparse` needs the time-signature and the tempo of the track (because measures are parsed separately), as well as the specification of a weighted grammar. We use a rhythm-oriented grammar similar to [24], detailed in appendix.

As described in [25], the root of the intermediate rhythmic tree is the first measure. Its left child is a tree describing its beat decomposition, and its right child is a node pointing at the root of the next measure. We rewrite this tree so that all measures are children of the same global root. A n-measures-long track will therefore have a root with n children. This rewriting allows us to reduce the maximum depth of the rhythmic tree, which would otherwise grow linearly with n. The resulting tree is shown on the right of Figure 2. In this tree, each node is labeled by the identifier of the associated production rule in the grammar. [2] Each leaf is a terminal symbol, labeled by the note and properties from the associated MIDI events, *i.e.* pitch [3] and velocity. Note that multiple instruments can be playing at the same time, so a leaf can be associated to several events.

As an example, in Figure 2, the first bar (red frame) is split in two sections, each of half note length (rule 5). Then, the first half gets split into two quarter-length sections (rule 7). The second child of this node, a quarter note, is split into four sixteenth notes (rule 11). Finally, each of those sixteenth notes leads to a terminal symbol (rule 0), with MIDI events attached to it, *e.g.*, the second child has two `NOTE_ON` events, respectively with pitch 36 and velocity 0.3, and with pitch 46 and velocity 0.5.

As we cannot directly feed the tree structure to the models, we first linearize it using a pre-order depth-first traversal: we start from root, and traverse the nodes recursively following the left-most child, only going back up when the current branch has been fully traversed. This produces a sequence of nodes containing the identifier of their rule in the grammar, as well as, in the case of leaves, the list of playing instruments and their velocity. We encode every node into a $d = (m + n)$-dimensional vector. m is the number of rules in the grammar, and the first part of the vector is the one-hot-encoded identifier of the production rule associated with the node. n is the number of possible instruments, and the second part of the vector contains the normalized velocity for each instrument. If an instrument is not playing for this note, its velocity is set to zero. For non-terminal rules, this second part is entirely zero. This linearization results in the matrix on the left of Figure 2, *i.e.* a sequence $S = \{s\}_{t \in [\![1,T]\!]}$ where $s_t \in \mathbb{R}^d$ is the vector associated to a node, and T is the total number of nodes. Therefore, our linearized rhythmic tree results in a multi-dimensional sequence S, that can be fed in all usual deep models such as RNN and Transformers.

Note that this representation is significantly shorter than tokenizations or piano rolls. In average, the sequences are only around 18% longer than note tuples, while containing much more information about the rhythm structure.

3.2 Tree-based Positional Encoding for Transformers

While RNN can model the position in the sequence through their hidden state, Transformers process sequences as a bag of words, without any positional information. To overcome this issue, positional encoding [26] was introduced to incorporate information about the position of an element in the Transformer model.

[2] See the ruleset with IDs in Section 2 of the supplementary material.

[3] In the case of drums, the "pitch" corresponds to the drum used, *e.g.* cymbal, snare, tom, etc.

Classical positional encoding [26] creates a vector PE of dimension d using sine and cosine functions of increasing frequencies:

$$\omega_{pos,i} = \frac{pos}{\tau^{(\frac{2i}{d})}}, \quad \begin{array}{l} \mathrm{PE}(pos, 2i) = \sin(\omega_{pos,i}) \\ \mathrm{PE}(pos, 2i+1) = \cos(\omega_{pos,i}) \end{array} \tag{1}$$

where pos is the position of the element in the sequence, d the size of the embedding, $i \in [\![1, d/2]\!]$ the dimension, and $\tau = 10000$ as in [26].

3.2.1 *Continuous Positional Encoding*

For musical data, this positional encoding is not related to the temporal organization of the notes. Depending on how the sequence S was built, the position pos of an element can be arbitrary, such as *e.g.* tokenizations where a note is split into several tokens for pitch, velocity and duration, or note tuples where two simultaneous notes can be interchanged. For encoding note tuples, we therefore introduce a continuous positional encoding that replaces the position in the sequence by the timestamp of the note in the track:

$$\omega_{t,i} = \frac{2\pi}{T_S} \cdot \frac{t}{(T_L/T_S)^{\frac{2i}{d}}}, \quad \begin{array}{l} \mathrm{PE}(t, 2i) = \sin(\omega_{t,i}) \\ \mathrm{PE}(t, 2i+1) = \cos(\omega_{t,i}) \end{array} \tag{2}$$

where t is the absolute starting time of the note in seconds and T_S and T_L are respectively the smallest and largest periods of the sine functions. This encoding allows two simultaneous notes to share the same positional encoding.

3.2.2 *Tree-based Positional Encoding*

A downside of linearizing the rhythmic tree is that we lose the explicit hierarchical structure between a parent node and its children. The structure is still implicitly encoded in the linearized sequence S in the rule identifiers, but the model would have to learn how the grammatical rules operate to rebuild the tree and leverage its structure.

To better represent the rhythmic tree, we use a hierarchical tree-based positional encoding (TBPE) that encodes the position of a node *in the tree*, rather than its position in the linearized sequence. Some TBPE have been proposed in the literature, *e.g.* for code translation to help Transformers process abstract syntax trees [27, 28]. Since our trees are bounded in depth at d_{max}, we associate to each node $\mathcal{N}$ a vector of size $2d_{\text{max}}$ that represents the path to a node from the root of the tree. This process is illustrated in Figure 3. Element k represents the index of the child traversed at depth k, while element $k + d_{\text{max}}$ is the total number of children of the parent node at depth k. For example, to reach node F, we go through node R (child #1 over 1), then node A (child #1 over 4), then node F (child #2 over 2). If the depth of the node $\mathcal{N}$ is less than d_{max}, then the remaining elements of the vector are padded with zeros. This makes explicit in the positional encoding the *parent* $\rightarrow$ *child* relations, along with depth and breadth properties. It becomes easier for the model to understand that notes can belong to a larger structures (*e.g.* triplet or four semiquavers).

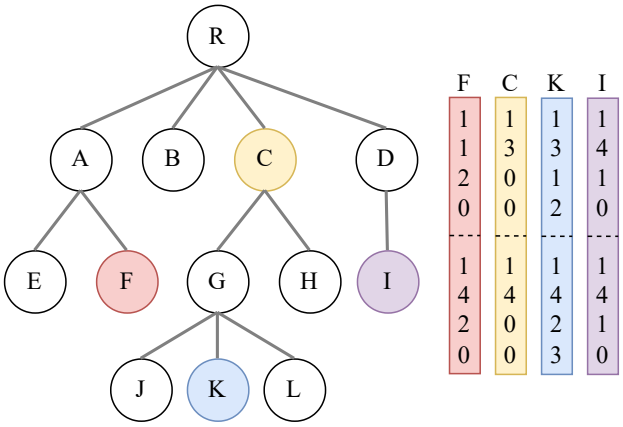


Figure 3: Example of tree-based positional encoding (TBPE) for a tree of maximum depth $d_{\text{max}} = 4$.

4. EXPERIMENTS AND RESULTS

4.1 Dataset and Task

Our models are trained and evaluated for style classification on the Groove MIDI Dataset (GMD) [6]. It consists in 13.6 hours of drumming music, played by humans with a metronome. Each track is labelled with a style provided by the drummer, alongside tempo and time-signature. The dataset is composed of long sequences (few minutes) and short beats and fills. We only consider long sequences, as short sequences are less representative of a specific style. We also discard non-4/4 tracks (around 1% of the dataset), as we use a 4/4 musical grammar, and a few tracks that `qparse` failed to parse [4]. We focus on the 4 most represented styles: funk, jazz, latin and rock. The final subset contains 326 tracks, representing 7.5 hours of drumming, split into the train/validation/test sets (80%/10%/10%) as the original dataset [6]. Each track is then further divided into multiple chunks of n measures with a sliding window.

4.2 Representations

In addition to our LRT, we evaluate common representations of MIDI data for style classification.

Piano Roll We sample the MIDI data at frequency f. We compare $f = 30\,\text{Hz} \approx 33.3\,\text{ms}$ per time step, as $30\,\text{ms}$ is considered as the simultaneity threshold for the human ear [29], and $f = 50\,\text{Hz} = 20\,\text{ms}$ per time step, to see if models would improve with finer granularity, at the expense of sequence length. Every time step is represented by a vector in $v \in [0, 1]^{22}$. Each dimension represents one of the 22 instruments of the drum kit. v_i encodes the velocity of the i-th instrument, normalized between 0 and 1 using maximum normalization. Note that the duration of notes in drums MIDI files is arbitrary, as only onset and velocity matter. All durations are set to $100\,\text{ms}$ in the Groove MIDI dataset. In our dataset, the average length of a piano roll is around 2455 for $f = 30\,\text{Hz}$, and 4092 for $f = 50\,\text{Hz}$.

Sequence of Tokens We experiment with various tokenizers from the literature, that quantify velocities and tim-

[4] As these tracks are only in the train and validation sets, this does not affect the fairness of the final comparison.

Representation used			LSTM			Transformer		
Type	**Variation**	**Avg. len.**	**Test F1 score**	**# params.**	**# bars**	**Test F1 score**	**# params.**	**# bars**
Piano roll	50 steps/second	4092	*0.618 ± 0.033*	576 522	4	0.545 ± 0.011	253 130	2
	30 steps/second	2455	**0.663 ± 0.023**	552 458	4	0.486 ± 0.026	20 330	4
Note Tuple	-	733	0.568 ± 0.024	555 530	8	0.492 ± 0.014	216 506	2
Tokenization	MIDI-Like	2767	0.565 ± 0.026	42 442	4	0.576 ± 0.011	360 586	8
	REMI	2502	0.475 ± 0.051	38 282	8	0.517 ± 0.028	17 626	4
	Structured	2646	0.599 ± 0.014	282 826	2	*0.598 ± 0.011*	232 162	8
	TSD	2464	0.487 ± 0.029	23 274	8	0.486 ± 0.032	20 178	4
LRT	Simple linearization	863	0.603 ± 0.014	1 358 346	8	0.556 ± 0.037	230 378	8
	With TBPE	863	0.596 ± 0.014	252 170	4	**0.660 ± 0.019**	88 138	4

Table 1: Performance of the different representations and model combinations on the GrooveMIDI dataset. We report macro F1 scores on the test set for the best model of each couple model/representation, alongside the model's number of parameters, the length (in bars) of input samples, and the average sequence length of each representation. Best results for each model type are in **bold**, second best in *italics*.

ings to limit the size of the vocabulary: MIDI-Like [13, 14], TSD [16], Structured [15] and REMI [17] tokenizers. We use the default parameters from [7], except for pitch range which is set to the min/max instrument ID from the GMD. Models trained on tokenizations use a 64-dimensional embedding, as recommended in [8]. Akin to piano rolls, tokenizers produce sequences with 2400 to 2800 elements.

Note Tuples We also consider the note tuples [18] representation that uses a single vector for each note. Each vector has 25 dimensions: the 22 one-hot-encoded instrument, followed by normalized velocity, note duration and time-shift to the previous note. This results in shorter sequences, with as many elements as there are notes. Average sequence has 733 elements, 3.5× less than tokenization methods.

Linearized Rhythmic Tree We use a simplified rhythm grammar of 15 rules on the GMD. As this grammar does not allow notes shorter than a 1/32nd note, the maximum depth $d_{\max}$ of a leaf in the rhythmic tree is 6. Although slightly longer than note tuples, the resulting sequences remain on the smaller side with an average of 863 elements.

4.3 Models

We chose to focus on sequential representations and therefore consider two popular architectures: LSTM [30] and Transformers [26]. The model inputs are fed as chunks of 2, 4 or 8 measures. We perform a hyperparameter search for the number of bars, number of layers and layer width on the validation set and retain the best architectures for each (model, representation) combination. As our grammar parser uses the track's tempo, we inject this information in non-grammatical models for a fair comparison by concatenating the tempo to the features vector in the last layer.

LSTM architecture We consider bidirectional LSTM models [31] and we experiment with a depth of 1 to 4 layers and a fixed width of 8 to 256 neurons per layer. Even though LSTMs do not require positional encoding, we also evaluate our LRT representation with TBPE to assess whether the explicit rhythmic structure is beneficial to the model.

Transformer architecture We use standard Transformers with an embedding layer, *i.e.* a linear projection, between the input and the first Transformer block. The models have 1 to 4 encoder layers, each with 2 to 16 attention heads. We also experiment with a feature size of 2 to 32 dimensions per head and 8 to 64 neurons in the feedforward network. We use the classical positional encoding for token sequences, the continuous positional encoding for piano rolls and note tuples, and either the classical or the tree-based positional encoding for LRTs. Regarding continuous encoding, we use $T_S = 100\,\text{ms}$ so that even close notes have a different encoding, and $T_L = 300\,\text{s}$, as temporal context is unlikely to matter beyond several minutes.

Final models are trained with a batch size of 128, using the AdamW optimizer [32] with a learning rate of 0.001, decayed by a factor 10 every 50 epochs with weight decay and dropout. Early stopping occurs when the validation F1 score plateaus with a patience of 200 epochs. Models are trained using the standard cross-entropy loss. To alleviate the class imbalance (185 rock tracks versus 50 for the other classes), we use class inverse median frequency weighing. We report the macro F1 scores averaged over all classes.

4.4 Main Results

We report in Table 1 the test scores of the best combinations from the hyperparameter search, averaged over five runs.

LSTM with 30 Hz piano rolls and Transformer with LRT/TBPE are the combinations that lead to the best F1 scores overall (≈ 0.66). The former is a 3-layer LSTM model, each composed of 64 neurons, performing on 2-bar-long samples. The latter is a 4-layer Transformer model, each using 2 heads with 32 features per head (so a 64-dimensional input vector), and a feedforward network of 32 neurons trained on 4-bar-long chunks. Although both models achieve comparable performance, note that the Transformer model needs 6× fewer parameters than the LSTM.

We observe that the TBPE provides important information for style classification. Transformer models using a

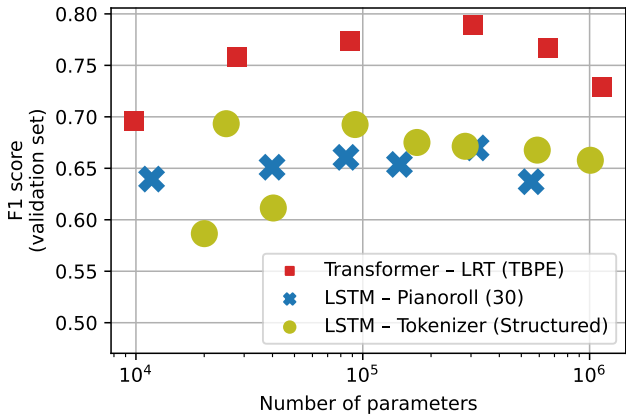


Figure 4: F1 scores on the validation set vs. number of parameters for a selected set of models. We observe that Transformers trained on LRT consistently outperform other models at similar capacity.

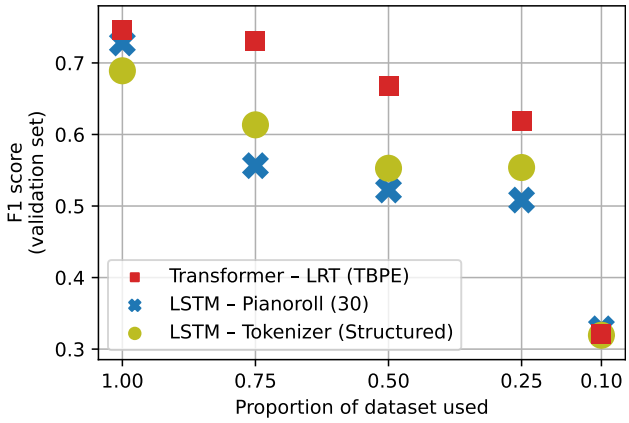


Figure 5: F1 scores on the validation set vs. percentage of training samples used. Transformers trained on LRT exhibit a less severe performance drop when the number of training samples decreases compared to existing models.

classical positional encoding achieve lower classification performance (≈ 0.56). Surprisingly, using TBPE is beneficial for LSTMs also: both our LSTM models trained on LRT achieve nearly identical F1 scores (≈ 0.6), however injecting the TBPE allows us to use a RNN with $5\times$ fewer parameters. This confirms that explicitly encoding the node position in the tree makes it easier for the models to understand the rhythmic structure of the track.

Finally, we observe that tokenization and note tuples tend to underperform overall. Structured MIDI tokenization achieves the best of tokenizer F1 score (≈ 0.6) both for LSTM and Transformer architecture, followed by MIDI-Like, however at the cost of a higher number of parameters. Token or note tuple sequences seem difficult to learn for the models. For RNN, we hypothesize that this is due to the regular sampling assumption made by these models. Each element is processed by the same recurrent loop, meaning that the model needs to learn the structure of the sequence, *e.g.* what each token represents. In comparison, piano rolls with a fixed time step where all elements represent the same object tend to have higher performances with LSTMs.

4.5 Model Parameter Efficiency

We evaluate some representative models by varying their capacity, *i.e.* number of parameters. More specifically, we experiment with 4, 8, 32 and 64 number of features per head for the Transformer, and 16, 32, 48, 64, 96 and 128 neurons in the hidden layers for LSTM. We report F1 scores on the validation set in Figure 4. We observe that, at comparable number of parameters, the Transformer trained on the LRT always lead to higher F1 scores than the compared models. This demonstrates that the rhythmic information embedded in our rhythmic tree not only results in shorter sequences, but also can be leveraged by smaller models for better or on par performance compared to existing works.

4.6 Training Samples Efficiency

Finally, we evaluate how representation affects the amount of data needed to train our models. We compare the same models as in Section 4.5 and train them with a random subset of 75%, 50%, 25% and 10% of the training set. F1 scores on the validation set are reported in Figure 5. We observe that the Transformer model trained with the linearized rhythmic tree and the tree-based positional encoding consistently outperforms the structured tokenizer and the piano roll. The performance drop between 100% and 75% is minimal, and overall the LRT-based Transformer degrades more gracefully when the number of training samples decreases compared to the other models. This underlines the relevance of the LRT, that encodes higher level musical information and better represents the invariance of musical style to spurious variations in the input MIDI file, such as slight changes in timings or velocity.

5. CONCLUSION AND FUTURE WORK

We evaluated different representations of MIDI data for drumming style classification. We introduced a new representation based on the linearization of a rhythmic tree obtained by parsing a MIDI file using a musical grammar. This representation provides richer features while being more compact than traditional piano rolls or tokenization strategies. Associated with a Transformer architecture using a tree-based positional encoding, we show that this representation achieves style classification performance on par with the best models from the literature with much fewer parameters. We also provide evidence that our representation is more resilient when trained on smaller datasets.

Future works involve extending this tree-based representation beyond homophonic input, *e.g.* for polyphonic piano pieces. Building the parsing tree could also be achieved on music scores, making it possible to directly classify scores at the mere symbolic level. In addition, we would like to evaluate this approach on more diverse tasks, as representation could be beneficial not only for discriminative models, but also for generative models, *e.g.* in music generation tasks, to produce syntactically correct performances with respect to the specified grammar [33].

6. ACKNOWLEDGEMENTS

We thank Florent Jacquemard for his work on `qparse`, fruitful discussions on the design of the rhythmic grammar for drums and advice throughout this project. Additional thanks are dedicated to Lydia Rodriguez de la Nava for her help in adapting `qparse` to drums rhythm parsing.

7. REFERENCES

[1] Z. Cataltepe, Y. Yaslan, and A. Sonmez, "Music genre classification using MIDI and audio features," *EURASIP Journal on Advances in Signal Processing*, vol. 2007, pp. 1–8, 2007.

[2] C. McKay and I. Fujinaga, "jSymbolic: A feature extractor for MIDI files." in *ICMC*, 2006.

[3] M. S. Cuthbert, C. Ariza, and L. Friedland, "Feature extraction and machine learning on symbolic music using the music21 toolkit." ser. Proceedings of the 12th International Society for Music Information Retrieval Conference, 2011, pp. 387–392.

[4] F. Simonetta, A. Llorens, M. Serrano, E. García-Portugués, and Á. Torrente, "Optimizing feature extraction for symbolic music," *Proceedings of the 24th International Society for Music Information Retrieval Conference*, 2023.

[5] Q. Kong, K. Choi, and Y. Wang, "Large-scale midi-based composer classification," *arXiv preprint arXiv:2010.14805*, 2020.

[6] J. Gillick, A. Roberts, J. Engel, D. Eck, and D. Bamman, "Learning to groove with inverse sequence transformations," in *International Conference on Machine Learning (ICML)*, 2019.

[7] N. Fradet, J.-P. Briot, F. Chhel, A. El Fallah Seghrouchni, and N. Gutowski, "MidiTok: A python package for MIDI file tokenization," in *Extended Abstracts for the Late-Breaking Demo Session of the 22nd International Society for Music Information Retrieval Conference*, 2021.

[8] H. Zhang, E. Karystinaios, S. Dixon, G. Widmer, and C. E. Cancino-Chacón, "Symbolic Music Representations for Classification Tasks: A Systematic Evaluation," Milan, Italy, pp. 848–858, Nov. 2023.

[9] G. Wiggins, E. Miranda, A. Smaill, and M. Harris, "A Framework for the Evaluation of Music Representation Systems," *Computer Music Journal*, vol. 17, Oct. 1993.

[10] B. Wang and Y.-H. Yang, "PerformanceNet: Score-to-Audio Music Generation with Multi-Band Convolutional Residual Network," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 1174–1181, Jul. 2019.

[11] F. Foscarin, K. Hoedt, V. Praher, A. Flexer, and G. Widmer, "Concept-Based Techniques for "Musicologist-friendly" Explanations in a Deep Music Classifier." [object Object], 2022.

[12] G. Velarde, T. Weyde, C. E. Cancino-Chacón, D. Meredith, and M. Grachten, "Composer Recognition Based on 2D-Filtered Piano-Rolls," in *International Society for Music Information Retrieval Conference*, Aug. 2016.

[13] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. M. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. Hoffman, M. Dinculescu, and D. Eck, "Music Transformer: Generating Music with Long-Term Structure," in *International Conference on Learning Representations*, Sep. 2018.

[14] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, no. 4, pp. 955–967, Feb. 2020.

[15] G. Hadjeres and L. Crestel, "The Piano Inpainting Application," *ArXiv*, Jul. 2021.

[16] N. Fradet, N. Gutowski, F. Chhel, and J.-P. Briot, "Byte pair encoding for symbolic music," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 2001–2020.

[17] Y.-S. Huang and Y.-H. Yang, "Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions," *Proceedings of the 28th ACM International Conference on Multimedia*, pp. 1180–1188, Oct. 2020.

[18] C. Hawthorne, A. Huang, D. Ippolito, and D. Eck, "Transformer-nade for piano performances," in *NIPS 2nd Workshop on Machine Learning for Creativity and Design*, 2018.

[19] J. E. Hopcroft and J. D. Ullman, "Introduction to automata theory, languages and computation," 1979. [Online]. Available: https://api.semanticscholar.org/CorpusID:31901407

[20] Y. Amagasu, F. Jacquemard, and M. Sakai, "Tokenization of MIDI Sequences for Transcription," in *9th International Conference on Technologies for Music Notation and Representation (TENOR 2024)*, Apr. 2024.

[21] C. Agon, K. Haddad, and G. Assayag, "Representation and rendering of rhythm structures," in *Second International Conference on Web Delivering of Music, 2002. WEDELMUSIC 2002. Proceedings.*, Dec. 2002, pp. 109–113.

[22] F. Jacquemard, P. Donat-Bouillud, and J. Bresson, "A Structural Theory of Rhythm Notation based on Tree Representations and Term Rewriting," in *Mathematics*

and Computation in Music: 5th International Conference, MCM 2015, vol. 9110. Springer, Jun. 2015, p. 12.

[23] F. Foscarin, F. Jacquemard, P. Rigaux, and M. Sakai, "A Parse-based Framework for Coupled Rhythm Quantization and Score Structuring," in *MCM 2019 - Mathematics and Computation in Music*, vol. Lecture Notes in Computer Science. Springer, Jun. 2019.

[24] M. Digard, F. Jacquemard, and L. Rodriguez-de la Nava, "Automated Transcription of Electronic Drumkits," in *4th International Workshop on Reading Music Systems (WoRMS)*, ser. Proceedings of the 4th International Workshop on Reading Music Systems, online, Spain, Nov. 2022.

[25] F. Jacquemard and L. Rodriguez de La Nava, "Symbolic Weighted Language Models, Quantitative Parsing and Automated Music Transcription," in *CIAA 2022 - International Conference on Implementation and Application of Automata*, ser. Lecture Notes in Computer Science, Vol 13266, P. Caron and L. Mignot, Eds. Rouen, France: Springer, Jun. 2022, pp. 67–79.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. ukasz Kaiser, and I. Polosukhin, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[27] V. Shiv and C. Quirk, "Novel positional encodings to enable tree-based transformers," in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019.

[28] H. Peng, G. Li, Y. Zhao, and Z. Jin, "Rethinking Positional Encoding in Tree Transformer for Code Representation," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 3204–3214.

[29] W. Goebl, "Melody lead in piano performance: Expressive device or artifact?" *The Journal of the Acoustical Society of America*, vol. 110, pp. 563–72, Aug. 2001.

[30] S. Hochreiter and J. Schmidhuber, "Long Short-term Memory," *Neural computation*, vol. 9, pp. 1735–80, Dec. 1997.

[31] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional LSTM networks," in *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, vol. 4, Jul. 2005, pp. 2047–2052 vol. 4.

[32] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations*, Sep. 2018.

[33] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, "Grammar Variational Autoencoder," in *Proceedings of the 34th International Conference on Machine Learning*. PMLR, Jul. 2017, pp. 1945–1954.